

Git Workflow for Solo Developers

PHB 243b: Branches vs. Direct Commits

2026-01-15

When working alone, you have flexibility in how you use Git. Should you commit directly to `main`, or use branches? Both approaches are valid—the right choice depends on your project.

Option 1: Direct Commits to Main

All work happens on a single branch. Simple and linear.

```
# Edit files, then:
git add analysis.R
git commit -m "Add regression model"
# Repeat
```

Pros	Cons
Simpler workflow	No clean way to abandon experiments
Linear, readable history	Work-in-progress clutters history
Fewer commands to learn	Hard to juggle multiple tasks

Best for: Homework, single scripts, small projects.

Option 2: Feature Branches

Create a branch for each piece of work, merge when complete.

```
git checkout -b try-mixed-model # Create and switch to branch
# ... experiment ...
git add .
git commit -m "Test mixed effects approach"

# If it works:
git checkout main
git merge try-mixed-model
git branch -d try-mixed-model # Clean up

# If it fails:
git checkout main
git branch -D try-mixed-model # Abandon experiment, main unchanged
```

Pros	Cons
Experiment without risk to main	More commands, more overhead
Work on multiple features in parallel	Merge commits add noise
Self-review before merging	Overkill for simple changes
Builds habits for collaboration	

Best for: Research projects, anything exploratory, code you'll share.

Recommendation by Project Type

Project	Suggested Workflow
Homework assignments	Main only
Course projects (team)	Always branch + pull requests
Solo research analysis	Main for routine; branch for experiments
R packages, tools	Always branch
Anything for publication	Branch habit is worth building

The Practical Middle Ground

Commit directly to `main` for routine, confident changes. Create a branch when you think: *“Let me try something that might not work.”*

- **Experiment succeeds** → merge the branch, delete it
- **Experiment fails** → delete the branch, `main` history stays clean

This gives you simplicity for daily work and safety for exploration.

Quick Reference

```
# Create and switch to new branch
git checkout -b experiment-name

# See all branches (* marks current)
git branch

# Switch between branches
git checkout main
git checkout experiment-name

# Merge branch into main
git checkout main
git merge experiment-name

# Delete branch (after merging)
git branch -d experiment-name

# Delete branch (abandon without merging)
git branch -D experiment-name
```

Key Insight

Branches are cheap in Git—creating one writes only 41 bytes. Don't hesitate to branch when uncertain. The cost of an unnecessary branch is near zero; the cost of a broken `main` is real.